his allows UX code and most
application logic code to be consumed by a smaller subset of users under real world scenarios before being made available to all users on the environment.

Currently a canary stage covers the following services

GitLab web
GitLab api
GitLab Git https
GitLab websockets
GitLab registry
GitLab pages
GitLab shell
Gitaly (see below)
Praefect (see below)

Currently a canary stage does not cover the following services (when using canary for these services, you are using the version running in the "main" stage)

Sidekiq
GitLab Agent for Kubernetes
Mailroom
Redis
PostgreSQL

How does Gitaly and Praefect work in a canary stage?

As Gitaly and Praefect nodes are randomly (with weighting) chosen to store Git data, it's not entirely possible to have a clean separation of Gitaly/Praefect nodes between stages.

What this means is that when using an environment, the Gitaly/Praefect code used (depending on the repository in question, see below) may be from either stage (main or canary), regardless of what stage you are using through the web frontend, or what you have selected in next.gitlab.com. What Gitaly/Prafect node
is used follows the following rules

If you are modifying an existing repository, you will use the Gitaly server that the shard is currently assigned to.
If you are forking a repository, you will stay on the same shard
Otherwise, rails will select a random shard using weighted probabilities based on the weights assigned in the storage configuration

In the production environment, the following well known groups/repos are run of Gitaly/Praefect nodes in the canary stage exclusively

https://gitlab.com/gitlab-org

Accessing the canary stage of an environment

The credentials you use and permissions you have in the "canary" stage of an environment is identical to the "main" stage of the environment.

At a technical level, access to the canary stage of an environment is accomplished in one of the following ways

- Manually setting the gitlabcanary cookie to true in whatever browser/tool you are using to talk to the environment
- Going to next.gitlab.com and at the toggle on the page, selecting "Next" instead of "Current". Note this changes which stage you are using (main or canary) across all environments.

The canary stage is also automatically used in the following circumstances (with caveats)

- 5% of traffic for the "main" stage is automatically instead sent to the "canary" stage of an environment if the gitlabcanary is not set which is the default for most users. If you wish to ensure your requests are never sent to canary, manually set the gitlabcanary cookie to false in whatever browser/tool you are using to talk to the environment or ensure that "current" is toggled on https://next.gitlab.com
- In the production environment only, the canary stage is forcibly enabled for all users visiting GitLab Inc. operated groups:
  - GitLab.com
  - GitLab.org
  - charts
  - Unless you choose to manually opt out of using Canary on next.gitlab.com
- The traffic share is not exactly 5%. The current baseline is 4.7% determined by HAProxy weights,
  but the actual amount depends on several factors including: Availability of zones in GCP,
  time of day, share of traffic to GitLab groups.

Confirming I am using the "canary" stage of an environment

The best method when using the UI is to enable the performance bar and look at the name of the Kubernetes pod service your page. If it starts with gitlab-cny (and has a baby chicken next to it), you are using the canary stage. There will also be the word "next" in a green box next to the GitLab logo in the top left.

How do I view logs specifically for the canary stage of an environment?

When looking at logs for an environment in Elasticsearch, you can add a filter for json.stage.keyword is cny and that will only show you logs from the canary stage of the service in the environment.

How do I view metrics specifically for the canary stage of an environment?

When looking at a metrics dashboard in dashboards.gitlab.net for an environment, choose cny from the drop down for stage at the top.

How do I change a feature flag for canary stage?

As the database (where feature flags are stored) is shared between main and canary stage, enabling a feature flag following the normal chatops process) for the environment will change it for both main and canary stages of an environment.

Some examples for the most commonly used environments are as follows

Feature flags on staging and staging-canary:

Enable: /chatops run feature set featureflagname true --staging
Disable: /chatops run feature set featureflagname false --staging

Feature flags on production and production-canary:

Enable /chatops run feature set featureflagname true
Disable /chatops run feature set featureflagname false

How do I get console access to the canary stage?

Currently the canary stage has no console access, you can standard console access process to access a console server in the environment running the main stage of the code only. Note that as the database is shared between stages, depending on what actions your perform in the console, it will affect the "canary" stage as well as the "main" stage.

Globally disabling the canary stage

The Infrastructure department teams can globally disable use of canary in an environment if it is deemed necessary. As this is very disruptive to the testing and deployment process, the typical process for doing starts with declaring an incident.

Disabling canary stage in an environment

The chatops command to disable canary in an environment is as follows

markdown
Disable production-canary
/chatops run canary --disable --production

Disable staging-canary
/chatops run canary --disable

Re-enabling canary stage in an environment

The chatops command to re-enable canary in an environment is as follows

markdown

Disable production-canary
/chatops run canary --enable --production

Disable staging-canary
/chatops run canary --enable

SECTION: engineering/infrastructure/environments/staging-ref.md

Staging Ref

Staging Ref is a sandbox environment used for pre-production testing of the latest Staging Canary code with full access to the environment and control over data.

Name	URL	Purpose	Deploy	Database	Terminal access	Slack channel
Staging Ref	staging-ref.gitlab.com	Pre-production testing	Frequently (Parallel to gstg-cny)	Separate and local	All engineers	staging-ref

Purpose

- Cover testing needs of Test Platform and Development teams in a production-like environment
- Admin testing access
  - Current Staging (gstg) has customer data which is a blocker for giving more access to Development and Test Platform teams.
- Testing different paid tiers
- Democratizing testing and better test data
- Better access to test accounts and wider permissions
- Performant sandbox environment for engineers

Environment information

- Geo is setup on Staging Ref with these configurations:
  - Staging Ref US site - primary - 3k Cloud Native Hybrid Reference Architecture environment - stateless components (Webservice, Sidekiq, NGINX) deployed to Google Kubernetes Engine cluster and the remaining stateful components installed to GCP virtual machines
  - Staging Ref EU site - secondary - 3k Reference Architecture full Omnibus environment
- Deployed with GitLab Environment Toolkit (GET) and Deployer
- SSL Certificates automated with Let's Encrypt
- Google OAuth gives access to environment for GitLab team members
- Outgoing email configured with Mailgun
- Advanced Search is configured with Elasticsearch and GET
- Ultimate license with Free paid plan by default
- Sentry configured for error reporting
- Snowplow tracking is enabled and collected to snowplow.trx.gitlab.net

Deployment process

Staging Ref deployment runs parallel to Staging Canary deployment. Deployer triggers a job in

Staging-Ref GET Config to update the environment. Notifications about new deployments are sent to the announcements Slack channel.

Staging Ref pipelines do not block the deployment. If there are any failures with deployment to gstg-ref, please reach out to @release-managers.

```
plantuml
@startuml staging-ref
left to right direction

card "Deployment" as deploy 667ab3

card "gstg-ref" as gstgref ffee9a {
 together {
  card "Deployer" as deployer 6a9be7
  card "Staging-Ref GET Config" as stgrefget FF8C00
 }
}

card "gstg-cny" as gstgcny ffee9a
card "gstg" as gstg ffd59a
card "gprd-cny" as gprdcny ffd500
card "gprd" as gprd 7966b3
card "QA" as gstgcnyqa ffa7db
card "QA" as gstgqa ffa7db
card "QA" as gprdcnyqa ffa7db

deploy -[554488]-> gstgref
deployer -[554488]-> stgrefget

deploy -[554488]-> gstgcny
gstgcny -[554488]-> gstgcnyqa
gstgcnyqa -[554488]-> gprdcny
gprdcny -[554488]-> gprdcnyqa
gprdcnyqa -[554488]-> gstg
gstg -[554488]-> gstgqa
gstgqa -[554488]-> gprd

@enduml
```

How to use Staging Ref

Staging Ref is a safe playground for engineers who want to test latest Staging(gstg-cny) code. Staging Ref has several advantages that allow it to be a full-fledged sandbox environment:

- Staging Ref deployments do not block the deployment process and can be tweaked or updated by any GitLab engineer. Hence GitLab engineers have wide permissions and full control over the environment.
- Environment follows 3k hybrid architecture, so it is more performant than existing

Staging(gstg) and could be used for load testing if needed.

To sign in to the environment, navigate to staging-ref.gitlab.com and use your GitLab Google account in 'Sign in with Google' option.

After signing in you can proceed using the environment as required. If destructive changes were done to the environment or it ended up in a bad state after testing, create a request to rebuild the environment. Please reach out to the staging-ref Slack channel or raise an issue in Staging-Ref GET Config. The process is automated with Staging-Ref GET Config and will take about an hour to finish.

Enable Feature Flags

ChatOps commands can be used to enable or disable Feature Flags on Staging Ref. You can run this command in the staging-ref Slack channel.

Admin access

To promote your user to Admin, please sign in as Admin using the Staging Ref credentials from 1Password Engineering vault. Then navigate to the Admin Area's Users page and edit your user's Access Level.

Note that Staging Ref environment is shared across all engineers. If you plan to perform changes to GitLab Admin settings, use the staging-ref Slack channel to communicate changes broadly.

SSH access and Rails console

If you have gcloud or kubectl set up locally, then follow Connect from your local terminal. If not, then you can opt to connect via your browser.

Connect via your browser

1. Get access to the GCP project gitlab-staging-ref
1. Visit gitlab-toolbox workload in the gitlab-staging-ref project
1. In the Managed pods section, click on the Name of a Running pod. For example, the Name looks like gitlab-toolbox-5955db475c-ng2xr.
1. Click the Kubectl dropdown near the top
1. Hover over Exec to reveal a sub menu
1. Click toolbox
1. A Cloud Shell should start up
1. Edit the command kubectl exec -it gitlab-toolbox-5955db475c-ng2xr -- bash (the toolbox will have a different suffix) to execute the bash command with the interactive and TTY options.
1. At this point, you should be logged in to the toolbox pod. For Rails console, run gitlab-rails console.
1. See Kubernetes cheat sheet for more

Connect from your local terminal

1. Navigate to the staging-ref cluster or to the staging-ref geo cluster

1. Click Connect
1. Copy the command and run it locally to get kubeconfig
1. Follow Kubernetes cheat sheet
1. Also see additional developer tools

Request access to GCP project and environment

If you need access to Staging Ref components in the GCP project(gitlab-staging-ref), please reach out in the staging-ref Slack channel. Someone can add you to gcp-staging-ref-sg@gitlab.com Google group.

As another option you can create an issue in the access-request project. Requests for access to server environments requires the approval of your manager and an Infrastructure manager.

Note that GitLab configuration changes will be overwritten by a new deployment to the environment. Environment updates can be locked if needed by a request to @release-managers in the staging-ref Slack channel.

A simplified process to request SSH access to Staging Ref virtual machines and the GKE cluster is being worked on in issue343938.

Trigger E2E test pipelines

The full or smoke E2E test suite may be triggered on demand in the staging-ref project. Results will also be posted to the staging-ref Slack channel.

Monitoring

Monitoring implementation was done in (epic594). Documentation can be found in the runbooks.

Dashboards for Staging Ref can be found in Grafana under the staging-ref folder. There are other existing dashboards which may also show Staging Ref information if you select environment=gstg-ref.

The Geo secondary site is running Grafana at <https://geo.staging-ref.gitlab.com/-/grafana>. Credentials can be found in EU site monitoring section in Staging Ref credentials in 1Password Engineering vault.

If you need a specific dashboard or an existing dashboard does not work please reach out to staging-ref channel.

Upgrade paid plans

By default, all users and groups are on the Free plan. To upgrade a paid plan use Admin account and do the following:

1. Navigate to Admin area.
1. Select Users or Groups section depending on what entity you would like to upgrade.
1. Search for user or group by name and click "Edit".
1. Select the required paid plan in "Plan".

1. Click "Save changes".

Watch this demo to see an example when a group was promoted to Premium plan.

Pre-existing test accounts

Staging Ref environment has pre-existing accounts that can be used for testing. For example, Admin accounts on different paid plans, Auditor user, QA users. All credentials are stored in Staging Ref credentials in 1Password Engineering vault.

Working with a SAML SSO enabled group

An Okta application has been setup to act as SAML idP for https://staging-ref.gitlab.com/groups/saml-sso-group.
Two users with names gitlab-qa-saml-sso-user1 and gitlab-qa-saml-sso-user2 have been created and added to Okta and assigned the application. These users are also available in staging-ref environment.

Please note that all credentials and values for fields mentioned below are saved in 1Password Engineering Vault in "Staging Ref credentials" under "User credentials for saml-sso-group Group".

For using SAML SSO, you will need to:

1. As an admin, create the group at https://staging-ref.gitlab.com/groups/saml-sso-group if it does not already exist.
1. Upgrade the pricing plan of this group to "Premium".
1. Visit https://staging-ref.gitlab.com/groups/saml-sso-group/-/saml and:

   - Check "Enforce SSO-only authentication for web activity for this group"
   - Update the value of "Identity provider single sign-on URL" to the value saved in 1Password
   - Update the value of "Certificate fingerprint" to the value saved in 1Password

1. Save the changes.
1. Sign out.

The first time you visit https://staging-ref.gitlab.com/groups/saml-sso-group and try to log in, you will be asked to sign in to GitLab with an existing account to link the SAML identity. Use username gitlab-qa-saml-sso-user1 or gitlab-qa-saml-sso-user2 to sign in. The credentials are in 1Password.

Future iterations and known limitations

Staging Ref environment has some known limitations that will be worked on:

- Test data configuration will be explored (epic7020)
- Configure Shared Runners (issue353284)
- Setup Kibana for Staging Ref(issue351816)
- Configure CustomersDot portal for Staging Ref (issue352594)
- Incoming email setup (issue348970)

- Configure Unified URL on Staging Ref (issue370312)
- Load testing (issue344223, issue344224)
- Increase Staging Ref adoption and gather feedback - (issue350744)
- Configure Pages on Staging Ref (issue383243)

Other outstanding work for Staging Ref is tracked in GitLab issue tracker.

Feedback

If you need some additional custom configuration for Staging Ref to be explored or you have other feedback and ideas for improvements, please reach out to eng-allocation-new-staging Slack channel or add a comment to the feedback issue350744.

SECTION: engineering/infrastructure/getting-assistance/_index.md

GitLab.com

If you need to report an incident - follow the instructions on the Report An Incident page.

If you are looking for help, and you know what service you need help with - find the owner in the tech stack. If they are listed below, then please create a Request For Help issue. If they aren't listed, please contact the owners through the Slack channel listed in the tech stack file.

If you need help, but you aren't sure who to ask, look through the teams below to see which team is the best fit for your question.

If you have read this whole page and are unsure how to proceed, please ask in the saas-platforms-help channel. You will be redirected to the appropriate team for help.

We aim to respond to your request within 24 hours. If you raise your request on a Friday, it may only be responded to on Monday.

Production Engineering

Teleport Requests

Requests for access via teleport should go exclusively in the teleport-requests channel. This type of request is responded to with best effort, but without a formal SLA. Please avoid using any other channel to escalate these requests, and do not directly ping the sre-oncall as they will not respond to these requests.

Observability

Open a request for help in the Request For Help Tracker

We can help with:

1. Observability
1. Logging
1. Metrics
1. Grafana / Kibana / Mimir / Prometheus
1. Error Budgets
1. Capacity Planning

Our Slack channel is: gobservability

Ops

Open a request for help in the Request For Help Tracker

We can help with:

1. Disaster Recovery
2. Incident Management
3. System and OS level patching

Our Slack channel is: ginfraops

Runway

Open a request for help in the Request For Help Tracker

We can help with:

1. Runway

Our Slack channel is: fgrunway

Foundations

Open a request for help in the Request For Help Tracker

We can help with:

1. Networking and traffic management (CDN / VPCs / DNS / Load Balancing / Service Discovery)
1. Rate Limiting: create an issue with the rate limiting request template
1. Cloudflare: create an issue with the Cloudflare Troubleshooting template
1. Kubernetes (K8s)
1. Config
1. Secrets Management with Vault
1. ops.gitlab.net and ops runners

Our Slack channel is: gfoundations

Cloud Connector

Open a request for help in the Request For Help Tracker

We can help with:

1. Cloud Connector
1. GitLab Duo Healthcheck problems

Our Slack channel is: gcloudconnector

Runners Platform

Open a request for help in the Request For Help Tracker

We can help with:

1. Hosted Runners questions (.com/Dedicated)
2. Pipelines and jobs troubleshooting
3. Runners related incident support

Our Slack channel is: grunnersplatform

Software Delivery

Delivery

Open a request for help in the Request For Help Tracker)

-We can help with:

1. Deployments to GitLab.com
1. Post Deployment Migrations (in relation to deployments)
1. Auto-Deploy
1. Hot Patching Process
1. Mean Time To Production
1. Release Management
1. Release Processes
1. Maintenance Policy
1. Patch Releases
1. Deployments
1. Monthly and Patch Releases
1. Backports

Our Slack channel is: gdelivery

Data Access

Durability

Open a request for help in the Request For Help Tracker)

We can help with:

1. Sidekiq
1. Redis
1. Gitaly Infrastructure
1. Backup / Restore
1. Disaster Recovery

Our Slack channel is: gdurability

 Dedicated

In order to deal with RFHs as efficiently as possible we have a number of issue templates. Please use the appropriate issue template for your request.

1. For a Private Link Config Request raise an issue in the Request For Help Tracker using the Private Link Request template
1. For a SAML Config Request raise an issue in the Request For Help Tracker using the SAML Config Request template
1. For a Switchboard Request for Help raise an issue in the Request For Help Tracker using the standard Dedicated Request template
1. For a standard request for help raise an issue in the Request For Help Tracker using the Switchboard Request template

We can help with:

1. Questions and support for GitLab Dedicated

Our Slack channel is: fgitlabdedicated

SECTION: engineering/infrastructure/incident-management/_index.md

{{% alert color="warning" %}}
If you're a GitLab team member and are looking to alert Reliability Engineering about an availability issue with GitLab.com, please find quick instructions to report an incident here: Reporting an Incident.
{{% /alert %}}

{{% alert color="warning" %}}
If you're a GitLab team member looking for who is currently the Engineer On Call (EOC), please see the Who is the Current EOC? section.
{{% /alert %}}

{{% alert color="warning" %}}
If you're a GitLab team member looking for the status of a recent incident, please see the incident board. For detailed information about incident status changes, please see the Incident Workflow section.
{{% /alert %}}

 Incident Management

Incidents are anomalous conditions that result in·or may lead
to·service degradation or outages. These events require human
intervention to avert disruptions or restore service to operational status.
Incidents are always given immediate attention.

The goal of incident management is to organize chaos into swift incident
resolution. To that end, incident management provides:

1. well-defined roles and responsibilities and workflow for members of the incident team,
1. control points to manage the flow information and the resolution path,
1. an incident review where lessons and techniques are extracted and shared

When an incident starts, the incident automation sends a message
in the correponding incident announcement channel
containing a link to a per-incident Slack channel for text based communication.
Within the incident channel, a per-incident Zoom link will be created.
Additionally, a GitLab issue will be opened in the Production tracker

Scheduled Maintenance

Scheduled maintenance that is a C1 should be treated as an undeclared incident.

30-minutes before the maintenance window starts, the Engineering Manager who is responsible for the change should notify the SRE on-call, the Release Managers and the CMOC to inform them that the maintenance is about to begin.

Coordination and communication should take place in the Situation Room Zoom so that it is quick and easy to include other engineers
if there is a problem with the maintenance.

If a related incident occurs during the maintenance procedure, the EM should act as the Incident Manager for the duration of
the incident.

If a separate unrelated incident occurs during the maintenance procedure, the engineers involved in the scheduled maintenance should
vacate the Situation Room Zoom in favour of the active incident.

Ownership

The Incident Lead role must be deliberately set for every incident. If you need help
determining the owner of an incident, the EOC can help.
The Incident Lead can delegate ownership to another engineer or escalate ownership to the IM at any time.
There is only ever one owner of an incident and only the owner of the incident can declare an incident resolved.
At anytime the Incident Lead can engage the next role in the hierarchy for support. The
Incident Lead role should always be assigned to the current owner.

Incident Management Structure

At GitLab, our incident management framework distinguishes between two important concepts:

1. Incident Response Roles: These are the functional positions needed during incident response, defined by specific responsibilities and actions, regardless of who fills them. Currently, we have three defined roles: Incident Lead, Incident Responder, and Communications Manager.

2. Response Teams: These are the specific teams and rotations responsible for staffing these roles. Different response teams may cover different environments (for example, GitLab.com vs. Dedicated) or specialized functions.

Understanding this distinction helps clarify who does what during incidents and ensures proper coordination across our incident management processes.

Incident Response Roles

Clear delineation of responsibilities is important during an incident. Quick resolution requires focus and a clear hierarchy for delegation of tasks. Preventing overlaps and ensuring a proper order of operations is vital to mitigation.

Role	Description	When Needed
Incident Lead	The owner of the incident who is responsible for the coordination of the incident response and will drive the incident to resolution. The Incident Lead should always be assigned the role in incident.io.	All incidents require an Incident Lead, which must be set purposefully per-incident. More information on choosing an Incident Lead can be found in the workflow section
Incident Responder	Performs technical investigation and mitigation. Responsible for the actual troubleshooting and resolving of the technical issues causing the incident.	All incidents
Communications Manager	Disseminates information to stakeholders and customers across multiple media. Manages external communications and status updates.	S1/S2 incidents or when significant communication is required

Response Teams

These are the specific teams that staff the incident response roles or provide additional support during incidents. Each team has specific on-call rotations and engagement processes.

Team	Role(s)/Function	Environment	Who?
Engineer On Call (EOC)	Primarily serves as the initial Incident Responder to automated alerting, and GitLab.com escalations - expectations for the role are in the Handbook for oncall. The checklist for the EOC is in our runbooks. There are runbooks designed to help EOC troubleshoot a broad range of issues - in the case where the runbooks are insufficient, the EOC will escalate by engaging the Incident Manager and CMOC.	GitLab.com	Generally an SRE and can declare an incident. Part of the "SRE 8 Hour" on call shift in PagerDuty.
Incident Manager On Call (IMOC)	Provides tactical coordination and leadership during complex incidents	GitLab.com	Rotation in PagerDuty
Communications Manager On Call (CMOC)	Staffs the Communications Manager role	All environments	Generally a member of the support team at GitLab.

Infrastructure Leadership	Provides escalation support for high severity incidents	All environments	A Staff+ or EM in the Infrastructure, Platform department.
Infrastructure Liaison	Communicates with executive team for S1 incidents	All environments	A grade 10+ member of Infrastructure.
IM Coordinator	Coordinates IMOC coverage and onboarding	GitLab.com	A single person dedicated to this function in the Infrastructure department
EOC Coordinator	Maintains EOC quality of life and confidence	GitLab.com	A single person dedicated to this function in the Infrastructure department

Role-Team Mapping

This table shows which teams typically fulfill which roles during incident response:

Role	Primary Team(s)	Alternative Team(s)
Incident Lead	Varies by incident type (see Incident Lead)	EOC, IMOC, Product Engineers
Incident Responder	EOC	Product Engineers, Other Subject Matter Experts
Communications Manager	CMOC	N/A

Detailed Role Responsibilities

Incident Lead Responsibilities

The Incident Lead is responsible for ensuring that the incident progresses and is kept updated. This role is not set automatically and should be assigned based on the type of incident. For more guidance on assigning Incident Lead, check out the workflow section. The Incident Lead should feel empowered to engage other parties such as the EOC or IMOC as necessary.

During the incident

1. Responsible for posting regular status updates using the /incident update in the incident Slack channel. These updates should summarize the current customer impact of the incident and actions we are taking to mitigate the incident. This is the most important section of the incident timeline. It will be referenced to status page updates and should provide a summary of the incident and impact that can be understood by the wider community.
2. Ensure that the incident issue has all of the required fields applied. If not set them using /incident field command from the incident slack channel
3. Ensure that the incident issue is appropriately restricted based on data classification, to mark the issue as confidential use /incident field and set the Keep GitLab Issue Confidential to true
4. The Incident Lead should not consider immediate work on an incident completed until the Incident Summary is filled out with useful information to describe all the key aspects of the Incident.
5. Ensuring that the Timeline section of the incident in the post-incident tab is accurate and complete with the start and end of the customer impact.
6. Ensuring that the root cause is stated clearly and plainly in the incident description by updating the causes section in the /incident summary, or can be alternatively shared as an internal status update using the :pushpin: (·) emoji or confidential follow-up if the root cause cannot be made public.
7. Ensure all follow-up items are properly documented and assign initial owners when possible.

After the incident

1. Review automatically created follow-up issues within one business day. Issues pasted into the incident channel are automatically linked as follow-ups, so it is possible some of these are not valid follow-up items.
2. Verify each follow-up issue has appropriate context from the incident.
3. Move follow-up issues from the follow-up issues project to the correct project for the responsible team (typically this will be gitlab-org/gitlab or production-engineering.
4. Apply appropriate labels such as team and group to follow-up issues.
5. The Incident Lead should review the comments and ensure that the corrective actions are added to the issue description, regardless of the incident severity.
6. For all Severity 1 and Severity 2 incidents, initiate an async incident review and inform the Engineering Manager of the team owning the root cause that they may need to initiate the Feature Change Lock process.

Incident Responder Responsibilities

An Incident Responder is anyone who contributes to the technical investigation and resolution of an incident. While the EOC team typically serves as primary responders, any GitLab team member with relevant expertise may be called upon to assist. Incident Responders should:

1. As an Incident Responder, your highest priority for the duration of your shift is the stability of GitLab.com.
2. When there is uncertainty of the cause of a degradation or outage, the first action of the Incident Responder is to evaluate whether any changes can be reverted. It is always appropriate to toggle (to previous state) any recently changed application feature flags without asking for permission and without hesitation. The next step is to review Change Requests and validate the eligibility criteria for application rollbacks.
3. The SSOT for who is the current EOC is the GitLab Production service definition in PagerDuty.
    1. SREs are responsible for arranging coverage if they will be unavailable for a scheduled shift. To make a request, send a message indicating the days and times for which coverage is requested to the eoc-general Slack channel. If you are unable to find coverage reach out to the EOC coordinator for assistance.
4. Alerts that are routed to PagerDuty require acknowledgment within 15 minutes, otherwise they will be escalated to the oncall Incident Manager.
    1. Alerts that page PagerDuty will automatically create a triage incident in incidents-dotcom-triage.
        1. If it is determined to be a true incident, the triage incident should be accepted by joining the channel and choosing "Accept it".
        1. The triage incident will automatically declined if no action is taken and the generating alert clears.
    1. Alert-manager alerts in alerts and feedalerts-general are an important source of information about the health of the environment and should be monitored during working hours.
    1. If the PagerDuty alert noise is too high, your task as an EOC is clearing out that noise by either fixing the system or changing the alert.
    1. If you are changing the alert, it is your responsibility to explain the reasons behind it and inform the next EOC that the change occurred.
    1. Each event (may be multiple related pages) should result in an issue in the production tracker. See production queue usage for more details.

5. If sources outside of our alerting are reporting a problem, and you have not received any alerts, it is still your responsibility to investigate. Declare a low severity incident and investigate from there.
    1. Low severity (S3/S4) incidents (and issues) are cheap, and will allow others a means to communicate their experience if they are also experiencing the issue.
    2. "No alerts" is not the same as "no problem"
6. GitLab.com is a complex system. It is ok to not fully understand the underlying issue or its causes. However, if this is the case, as Incident Responder you should page the IMOC to find a team member with the appropriate expertise. Requesting assistance does not mean relinquishing your responsibility.
7. As soon as an S1/S2 incident is declared, join the Zoom room for the incident. The Zoom link is in the bookmarks of the relevant incident channel.
    1. GitLab works in an asynchronous manner, but incidents require a synchronous response. Our collective goal is high availability of 99.95% and beyond, which means that the timescales over which communication needs to occur during an incident is measured in seconds and minutes, not hours.
8. Keep in mind that a GitLab.com incident is not an "infrastructure problem". It is a company-wide issue, and as Incident Responder, you are leading the response on behalf of the company.
    1. If you need information or assistance, engage with Engineering teams. If you do not get the response you require within a reasonable period, escalate through the IMOC.
    2. As Incident Responder, require that those who may be able to assist to join the Zoom call and ensure that they post their findings in Slack and pin (·) the message to the incident timeline.
9. By acknowledging an incident in PagerDuty, you are implying that you are working on it. To further reinforce this acknowledgement, post a note in Slack that you are joining the incident Zoom as soon as possible.
10. Be inquisitive. Be vigilant. If you notice that something doesn't seem right, investigate further.

Communications Manager Responsibilities

For serious incidents that require coordinated communications across multiple channels, the IMOC will rely on the Communications Manager for the duration of the incident.

The GitLab support team staffs an oncall rotation (CMOC) via the Incident Management - CMOC service in PagerDuty. They have a section in the support handbook for getting new CMOC people up to speed.

During an incident, the Communications Manager will:

1. Be the voice of GitLab during an incident by updating our end-users and internal parties through updates to our status page hosted by Status.io.
    - Tip: use /woodhouse incident post-statuspage on Slack to create an incident on Status.io. Any updates to the incident will have to be done manually by following these instructions.
2. Update the status page at regular intervals in accordance with the severity of the incident.
3. Notify GitLab stakeholders (customer success and community team) of current incident and reference where to find further information. Provide additional update when the incident is mitigated.
4. Given GitLab's directive to err on the side of declaring incidents early and often, it is important for the Communications Manager not to make public communications without first

confirming with the Incident Responder and IMOC that the incident has significant external customer impact. Rushing to communicate incidents before understanding impact can lead to a public perception of reliability impacts that may not be accurate, because we regularly declare an incident at Severity 1 or 2 initially and then downgrade it one or even two levels once the scope of customer impact is more clearly understood.

Detailed Team Responsibilities

Engineer On Call (EOC) Responsibilities

The Engineer On Call typically serves as the primary Incident Responder and is responsible for the mitigation of impact and resolution to the incident that was declared. The EOC should reach out to the IMOC for support if help is needed or others are needed to aid in the incident investigation.

1. As an EOC, your highest priority for the duration of your shift is the stability of GitLab.com.
2. When there is uncertainty of the cause of a degradation or outage, the first action of the EOC is to evaluate whether any changes can be reverted. It is always appropriate to toggle (to previous state) any recently changed application feature flags without asking for permission and without hesitation. The next step is to review Change Requests and validate the eligibility criteria for application rollbacks.
3. The SSOT for who is the current EOC is the GitLab Production service definition in PagerDuty.
    1. SREs are responsible for arranging coverage if they will be unavailable for a scheduled shift. To make a request, send a message indicating the days and times for which coverage is requested to the eoc-general Slack channel. If you are unable to find coverage reach out to the EOC coordinator for assistance.
4. Alerts that are routed to PagerDuty require acknowledgment within 15 minutes, otherwise they will be escalated to the oncall Incident Manager.
    1. Alerts that page PagerDuty will automatically create a triage incident in incidents-dotcom-triage.
        1. If it is determined to be a true incident, the triage incident should be accepted by joining the channel and choosing "Accept it".
        2. The triage incident will automatically declined if no action is taken and the generating alert clears.
        3. If there are multiple pages/triage incidents created for the same incident, merge them into the primary incident. However, resolving the incident takes precedence. It is fine if a related triage incident auto-closes instead of getting merged while you're working an incident.
5. If sources outside of our alerting are reporting a problem, and you have not received any alerts, it is still your responsibility to investigate. Declare a low severity incident and investigate from there.
    1. Low severity (S3/S4) incidents (and issues) are cheap, and will allow others a means to communicate their experience if they are also experiencing the issue.
    2. "No alerts" is not the same as "no problem"
6. GitLab.com is a complex system. It is ok to not fully understand the underlying issue or its causes. However, if this is the case, as EOC you should page the IMOC to find a team member with the appropriate expertise. Requesting assistance does not mean relinquishing your responsibility.

7. As soon as an S1/S2 incident is declared, join the Zoom room for the incident. The Zoom link is in the bookmarks of the relevant incident channel.
    1. GitLab works in an asynchronous manner, but incidents require a synchronous response. Our collective goal is high availability of 99.95% and beyond, which means that the timescales over which communication needs to occur during an incident is measured in seconds and minutes, not hours.
8. Keep in mind that a GitLab.com incident is not an "infrastructure problem". It is a company-wide issue, and as EOC, you are leading the response on behalf of the company.
    1. If you need information or assistance, engage with Engineering teams. If you do not get the response you require within a reasonable period, escalate through the IMOC.
    2. As EOC, require that those who may be able to assist to join the Zoom call and ensure that they post their findings in Slack and pin (·) the message to the incident timeline.
9. By acknowledging an incident in PagerDuty, you are implying that you are working on it. To further reinforce this acknowledgement, post a note in Slack that you are joining the incident Zoom as soon as possible.
10. Be inquisitive. Be vigilant. If you notice that something doesn't seem right, investigate further.

Incident Manager On Call (IMOC) Responsibilities

For general information about how shifts are scheduled and common scenarios about what to do when you have PTO or need coverage, see the Incident Manager onboarding documentation

When paged, the IMOC has the following responsibilities during a Sev1 or Sev2 incident and should be engaged on these tasks immediately when an incident is declared:

1. In the event of an incident which has been triaged and confirmed as a clear Severity 1 impact:
    1. Notify Infrastructure Leadership by typing /incident escalate in Slack. In the On-call teams drop-down menu, select dotcom leadership escalation with the appropriate message in the Notification Message. This notification should happen 24/7.
    2. In the case of a large scale outage where there is a serious disruption of service, the IMOC should check in with Infrastructure Leadership whether a senior member should be brought into the incident to coordinate and manage recovery efforts. This is to ensure that the person in charge of coordinating multiple parallel recovery efforts has a deeper understanding of what is required to bring services back online.
2. Consider engaging the release-management team if a code change related issue is identified as a potential cause and we need to explore rollbacks or expedited deployment. This can be done by using their slack handle release-managers
3. Ensure that necessary public communications are made accurately and in a timely fashion by the Communications Manager. Be mindful that, due to the directive to err on the side of declaring incidents early and often, we should first confirm customer impact with the Incident Responder prior to approving customer status updates.
4. If necessary, help the Incident Responder to engage development using the InfraDev escalation process.
5. If applicable, coordinate the incident response with business contingency activities.
6. Following the first significant Severity 1 or 2 incident for a new member of the IMOC, schedule a feedback coffee chat with the Engineer On Call, Communications Manager On Call, and (optionally) any other key participants to receive actionable feedback on your engagement.

The IMOC is the DRI for all of the items listed above, but it is expected that they will do it with the support of the Incident Lead, Incident Responder, or others who are involved with the incident. If an incident runs beyond a scheduled shift, the IMOC is responsible for handing over to the incoming IMOC member.

The IMOC won't be engaged on these tasks unless they are paged, which is why the default is to page them for all Sev1 and Sev2 incidents. In other situations, page the IMOC to engage them.

Infrastructure Leadership Responsibilities

To page the Infrastructure Leadership directly, run /inc escalate and choose the dotcom leadership escalation from the Oncall Teams drop-down menu

The Infrastructure Leadership is on the escalation path for both Engineer On Call (EOC) and Incident Manager On Call (IMOC).
This is not a substitute or replacement for the active IMOC (unless the current IMOC is unavailable).

They will be paged in the following circumstances:

1. If IMOC is unable to respond to a page within 15 minutes.
2. If there are multiple ongoing incidents that is overloading the EOC, or if coordination is required among multiple SREs, the Infrastructure Leadership can be paged to help coordinate recovery and bring in additional help if needed.

When paged, the Infrastructure Leadership will:

1. Join the incident call
2. Ask the Incident Responder if help is needed from additional SREs.
3. Ask the IMOC to ensure they are able to fulfill their duties.
4. Evaluate whether a separate zoom should be created for technical investigations.
5. Be the primary technical point of contact for the IMOC/CMOC to ensure the Incident Responder can focus completely on remediation.

Infrastructure Liaison Responsibilities

To page the Infrastructure Liaison directly, run /pd trigger and choose the Infrastructure Liaison as the impacted service.

During a verified Severity 1 Incident the IMOC will page the Infrastructure Liaison.
This is not a substitute or replacement for the active IMOC.

When paged, the Infrastructure Liaison will:

1. Make an overall evaluation of the incident and further validation of Severity.
2. Assist with further support from other teams, including those outside of Engineering (as appropriate)
3. Post a notice to e-group slack channel. This notice does not have to be expedited, but should occur once there is a solid understanding of user impact as well as the overall situation and current response activities. The e-group notice should be in the format below.

4. After posting the notice, continue to engage with the incident as needed and also post updates to a thread of the e-group notification when there are material/significant updates.

markdown
:s1: Incident on GitLab.com
· Summary ·
(include high level summary)
· Customer Impact ·
(describe the impact to users including which service/access methods and what percentage of users)
· Current Response ·
(bullet list of actions)
· Production Issue ·
Main incident: (link to the incident)
Slack Channel: (link to incident slack channel)

Team Coordinators

Incident Manager Coordinator

1. Around the 1st Tuesday of each month:
   - The coordinator will review any open ~IM-Onboarding::Ready and ~IM-Offboarding issues on the IM onboarding/offboarding board and add these team members to the schedule.
   - The schedule is updated by creating an MR that updates imlocals.tf.
     A single MR is created for all users that need updating. They will be assigned to the appropriate shift and the startdate will be set to the beginning of the second full month after the current day.
     For example, if the current day is Jan 5, the startdate will be make to go into effect during the first week of March.
2. An announcement will be posted in im-general indicating that the schedule has been modified with a link to the MR and a brief overview of who was added or removed.
3. All issues on the IM onboarding/offboarding board need to be reviewed once a month for overdue due dates.
   If any issues are overdue, the coordinator will need to check in with the author to see if they need more time or support to finish their on-boarding.

Engineer on-call Coordinator

The EOC Coordinator is focused on improving SRE on-call quality of life and setting up processes to keep on-call engineers across the entire company operating at a high level of confidence.

Responsibilities of this role:

1. Identifying gaps in process and tooling that help EOC increase QoL, capture via PI.
2. Coordinating regular training and workshops.
3. Enabling knowledge transfer between SREs as a follow up to incident reviews and notable incidents.
4. Facilitate larger changes regarding incident management through coordination and priority

setting with other teams inside of SaaS Platforms.

The EOC Coordinator will work closely with the Ops Team on core on-call and incident management concerns, and engage other teams across the organization as needed.

Tier 2 Oncall

Tier 2 on-calls are established to provide subject matter expertise when required. Additional teams may be added when appropriate.
To initiate onboarding of a new tier 2 team, follow the guidelines in Tier 2 Oncall Onboarding for teams
To page a Tier 2 team, use /inc escalate in Slack and choose the team you'd like to contact.

References

Other escalations

Further support is available from the Scalability and Delivery Groups if required.
Scalability leadership can be reached via PagerDuty Scalability Escalation (further details available on their team page).
Delivery leadership can be reached via PagerDuty. See the Release Management Escalation steps on the Delivery group page.

Incident Mitigation Methods - EOC/Incident Manager

1. If wider user impact has been established during an S1 or S2 incident, as EOC you have the authority - without requiring further permission - to Block Users as needed in order to mitigate the incident. Make sure to follow Support guidelines regarding Admin Notes, leaving a note that contains a link to the incident, and any further notes explaining why the user is being blocked.
    1. If users are blocked, then further follow-up will be required. This can either take place during the incident, or after it has been mitigated, depending on time-constraints.
        1. If the activity on the account is considered abusive), report the user to Trust and Safety so that the account can be permanently blocked and cleaned-up. Depending on the nature of the event, the EOC may also consider reaching out to the SIRT team.
        1. If not, open a related confidential incident issue and assign it to CMOC to reach out to the user, explaining why we had to block their account temporarily.
        1. If the EOC is unable to determine whether the user's traffic was malicious or not, please engage the SIRT team to carry out an investigation.

When to Engage an Incident Manager?

If any of the following are true, it would be best to engage an Incident Manager:

1. There is a S1/P1 report or security incident.
1. An entire path or part of functionality of the GitLab.com application must be blocked.
1. Any unauthorized access to a GitLab.com production system
1. Two or more S3 or higher incidents to help delegate to other SREs.

Please note that when an incident is upgraded in severity (for example from S3 to S1), incident.io automatically pages the EOC, IMOC, and CMOC via PagerDuty.

What happens when there are simultaneous incidents?

Occasionally we encounter multiple incidents at the same time. Sometimes a single Incident Manager can cover multiple incidents. This isn't always possible, especially if there are two simultaneous high-severity incidents with significant activity.

When there are multiple incidents and you decide that additional incident manager help is required, take these actions:

1. Post a slack message in im-general as well as the appropriate incident announcement channel asking for additional Incident Manager help.
1. If your ask is not addressed via slack, escalate to Infrastructure Leadership in PagerDuty.

Weekend Escalations

EOCs are responsible for responding to alerts even on the weekends. Time should not be spent mitigating the incident unless it is a ~severity::1 or ~severity::2. Mitigation for ~severity::3 and ~severity::4 incidents can occur during normal business hours, Monday-Friday. If you have any questions on this please reach out to an Infrastructure Engineering Manager.

If a ~severity::3 and ~severity::4 occurs multiple times and requires weekend work, the multiple incidents should be combined into a single severity::2 incident.
If assistance is needed to determine severity, EOCs and Incident Managers are encouraged to contact Reliability Leadership via PagerDuty

Incident Manager Escalation

A page will be escalated to the Incident Manager (IM) if it is not answered by the Engineer on Call (EOC).
This escalation will happen for all alerts that go through PagerDuty, which includes lower severity alerts.
It's possible that this can happen when there is a large number of pages and the EOC is unable to focus on acknowledging pages.
When this occurs, the IM should reach out in Slack in the corresponding incident announcement channel to see if the EOC needs assistance.

Example:

```plaintext
@sre-oncall, I just received an escalation. Are you available to look into LINKTOPAGERDUTYINCIDENT, or do you need some assistance?
```

If the EOC does not respond because they are unavailable, you should escalate the incident using the PagerDuty application, which will alert Infrastructure Engineering leadership.

How to Engage Response Teams

If during an incident, you need to engage the Incident Responder (EOC), IMOC, or Communications Manager (CMOC), page the person on-call using one of the following methods. This triggers a PagerDuty incident and pages the appropriate person based on the Impacted Service that you select.

- Use the /inc escalate command in Slack, select the correct team from the Oncall team drop down menu based on the team below,

or

- Directly from PagerDuty, navigate to Incidents page in PagerDuty, create a new incident and select the Impacted Service based on the team below.

Team to Page	Service Name
dotcom EOC	dotcom EOC
dotcom IMOC	dotcom IMOC
CMOC	dotcom CMOC

Incidents requiring direct customer interaction

If, during an S1 or S2 incident, it is determined that it would be beneficial to have a synchronous conversation with one or more customers a new Zoom meeting should be utilized for that conversation. Typically there are two situations which would lead to this action:

1. An incident which is uniquely impacting a single, or small number, of customers where their insight into how they are using GitLab.com would be valuable to finding a solution.
1. A large-scale incident, such as a multi-hour full downtime or regional DR event, when it is desired to have synchronous conversation with key customers, typically to provide another form of update or to answer further questions.

Due to the overhead involved and the risk of detracting from impact mitigation efforts, this communication option should be used sparingly and only when a very clear and distinct need is present.

Implementing a direct customer interaction call for an incident is to be initiated by the current Incident Manager by taking these steps:

1. Identify a second Incident Manager who will be dedicated to the customer call. If not already available in the incident, announce the need in im-general with a message like /here A second incident manager is required for a customer interaction call for XXX.
1. Page the Infrastructure Leadership pagerduty rotation for additional assistance and awareness.
1. Identify a Customer Success Manager who will act as the primary CSM and also be dedicated to the customer call. If this role is not clear, also refer to Infrastructure Leadership for assistance.
1. Request that both of these additional roles join the main incident to come up to speed on the incident history and current status. If necessary to preserve focus on mitigation, this information sharing may be done in another Zoom meeting (which could then also be used for the customer conversation)

After learning of the history and current state of the incident the Engineering Communications

Lead will initiate and manage the customer interaction through these actions:

1. Start a new Zoom meeting - unless one is already in progress - invite the primary CSM.
1. The Engineering Communications Lead and CSM should appropriately set their Zoom name to indicate GitLab, as well as their Role, CSM Engineering Communications Lead
1. Through the CSM, invite any customers who are required for the discussion.
1. The Engineering Communications Lead and the Incident Manager need to prioritize async updates that will allow for the correct information to flow between conversations. Consider using the incident slack channel for this but agree before the customer call starts.
1. Both the Engineering Communications Lead and CSM should remain in the Zoom with the customers for the full time required for the incident. To avoid loss of context, neither should "jump" back and forth from the internal incident Zoom and the customer interaction Zoom.

In some scenarios it may be necessary for most all participants of an incident (including the EOC, other developers, etc.) to work directly with a customer. In this case, the customer interaction Zoom shall be used, NOT the Incident Zoom. This will allow for the conversation (as well as text chat) while still supporting the ability for primary responders to quickly resume internal communications in the Incident Zoom.

Corrective Actions

Corrective Actions (CAs) are work items that we create as a result of an incident.
Only issues arising out of an incident should receive the label ~"corrective action".
They are designed to prevent the same kind of incident or improve the time to mitigation and as such are part of the Incidence Management cycle.
Corrective Actions must be related to the incident issue to help with downstream analysis.

Corrective Actions issues in the Production Engineering project should be created using the Corrective Action issue template to ensure consistency in format, labels and application/monitoring of service level objectives for completion

Issues that have the ~"corrective action" label will automatically have the ~"infradev" label applied.
This is done so teams these issues are follow the same process we have for development to resolve them in specific time-frames.
For more details see the infradev process.

Best practices and examples, when creating a Corrective Action issue

- Use SMART criteria: Specific, Measurable, Achievable, Relevant and Time-bounded.
- Link to the incident they arose from.
- Assign a Severity label designating the highest severity of related incidents.
- Assign a priority label indicating the urgency of the work. By default, this should match the incident Severity
- Assign the label for the associated affected service if applicable.
- Provide enough context so that any engineer in the Corrective Action issue's project could pick up the issue and know how to move forward with it.
- Avoid creating Corrective Actions that:
  - Are too generic (most typical mistake, as opposed to Specific)
  - Only fix incident symptoms.

  - Introduce more human error.
  - Will not help to keep the incident from happening again.
  - Can not be promptly implemented (time-bounded).
- Examples: (taken from several best-practices Postmortem pages)

Badly worded	Better
Fix the issue that caused the outage	(Specific) Handle invalid postal code in user address form input safely
Investigate monitoring for this scenario	(Actionable) Add alerting for all cases where this service returns >1% errors
Make sure engineer checks that database schema can be parsed before updating	(Bounded) Add automated presubmit check for schema changes
Improve architecture to be more reliable	(Time-bounded and specific) Add a redundant node to ensure we no longer have a single point of failure for the service

Runbooks

Runbooks are available for
engineers on call. The project README contains links to checklists for each
of the above roles.

In the event of a GitLab.com outage, a mirror of the runbooks repository is available on the Ops instance at https://ops.gitlab.net/gitlab-com/runbooks.

Who is the Current EOC?

Use the @sre-oncall handle to check who the current EOC is

When to Contact the Current EOC

The current EOC can be contacted via the @sre-oncall handle in Slack, but please only use this handle in the following scenarios.

1. You need assistance in halting the deployment pipeline. note: this can also be accomplished by Reporting an Incident and setting the custom field "Blocks Deployments" to "Yes".
1. You are conducting a production change via our Change Management process and as a required step need to seek the approval of the EOC.
1. For all other concerns please see the Getting Assistance section.

The EOC will respond as soon as they can to the usage of the @sre-oncall handle in Slack, but depending on circumstances, may not be immediately available. If it is an emergency and you need an immediate response, please see the Reporting an Incident section.

Reporting an Incident

If you are a GitLab team member and would like to report a possible incident related to GitLab.com and have the EOC paged in to respond, choose one of the reporting methods below. Regardless of the method chose, please stay online until the EOC has had a chance to come online and engage with you regarding the incident. Thanks for your help!

Report an Incident via Slack

Type /incident or /inc in GitLab's Slack and follow the prompts to open an incident issue.
It is always better to err on side of choosing a higher severity, and declaring an incident for a production issue, even if you aren't sure.
Reporting high severity bugs via this process is the preferred path so that we can make sure we engage the appropriate engineering teams as needed.

Incident Declaration Slack window

Field	Description
Name	Give a short description of what is happening. If you'd like to, you can leave it blank and change it later
Incident Type	Select the appropriate incident type: GitLab.com, Dedicated, SIRT, or Gameday depending on the service affected
Initial status	Choose "Active incident" if you've confirmed there's a problem and you'd like to investigate it right away, or "Triage a problem" for initial investigation
Severity	If unsure about the severity, but you are seeing a large amount of customer impact, please select S1 or S2. More details here: Incident Severity.
Summary (optional)	Provide your current understanding of what happened in the incident and the impact it had. It's fine to go into detail here
Who should be able to see this incident?	Choose "Everyone (public)" - this means everyone in this Slack workspace will have access. Choose "Private" to mark the issue confidential - do this for all security related issues or incidents that primarily contain information that is not SAFE.

Incident Declaration Results

As well as opening a GitLab incident issue, a dedicated incident Slack channel will be opened. incident.io will post links to all of these resources in the corresponding incident announcement channel. Please join the incident Slack channel, created and linked as a result of the incident declaration, to discuss the incident with the on-call engineer.

Definition of Outage vs Degraded vs Disruption and when to Communicate

This is a first revision of the definition of Service Disruption (Outage), Partial Service Disruption, and Degraded Performance per the terms on Status.io.
Data is based on the graphs from the Key Service Metrics Dashboard

Outage and Degraded Performance incidents occur when:

1. Degraded as any sustained 5 minute time period where a service is below its documented Apdex SLO or above its documented error ratio SLO.
1. Outage (Status = Disruption) as a 5 minute sustained error rate above the Outage line on the error ratio graph

In both cases of Degraded or Outage, once an event has elapsed the 5 minutes, the Engineer on Call and the Incident Manager should engage the CMOC to help with external communications. All incidents with a total duration of more than 5 minutes should be publicly communicated as quickly as possible (including "blip" incidents), and within 1 hour of the incident occurring.

SLOs are documented in the runbooks/rules

To check if we are Degraded or Disrupted for GitLab.com, we look at these graphs:

1. Web Service
   - Error Ratio
   - Apdex
1. API Service
   - Error Ratio
   - Apdex
1. Git service(public facing git interactions)
   - Error Ratio
   - Apdex
1. GitLab Pages service
   - Error Ratio
   - Apdex
1. Registry service
   - Error Ratio
   - Apdex
1. Sidekiq
   - Error Ratio
   - Apdex

A Partial Service Disruption is when only part of the GitLab.com services or infrastructure is experiencing an incident. Examples of partial service disruptions are instances where GitLab.com is operating normally except there are:

1. delayed CI/CD pending jobs
1. delayed repository mirroring
1. high severity bugs affecting a particular feature like Merge Requests
1. Abuse or degradation on 1 gitaly node affecting a subset of git repos. This would be visible on the Gitaly service metrics

High Severity Bugs

In the case of high severity bugs, we prefer that an incident is still created via Reporting an Incident. This will give us an incident issue on which to track the events and response.

In the case of a high severity bug that is in an ongoing, or upcoming deployment please follow the steps to Block a Deployment.

Security Incidents

If an incident may be security related, engage the Security Engineer on-call by using /security in Slack. More detail can be found in Engaging the Security Engineer On-Call.

Communication

Information is an asset to everyone impacted by an incident. Properly managing the flow of information is critical to minimizing surprise and setting expectations. We aim to keep interested stakeholders apprised of developments in a timely fashion so they can plan appr